PLAN COMPLETO: Sistema de Checkpoint + Warm Start Manual

Objetivo General

Crear un sistema que permita al agent (y al usuario):

1. **Guardar estado** de búsqueda MCTS después de cada experimento
2. **Cargar estado anterior** y continuar búsqueda (feature activable/desactivable)
3. **Ingresar expresión manualmente** en config.json para que ND2 arranque ahí
4. **Sin checkpoint**: Pierde info anterior (comportamiento normal)

Arquitectura General

```
config.json
├─ "use_checkpoint": true/false      ← Control global
├─ "checkpoint_mode": "auto" | "manual"
│  ├─ "auto": Continúa desde iteración anterior
│  └─ "manual": Usa expresión definida en config
└─ "initial_expression": "((x*(2.01*p_curr))-p_prev1))"  ← Campo manual

experiments/legendre/
├─ results/
│  ├─ iter001/
│  │  ├─ mcts_checkpoint.pkl  ← Guardar aquí (si enabled)
│  │  ├─ metrics.json
│  │  └─ data.json
│  ├─ iter002/
│  │  └─ mcts_checkpoint.pkl
│  └─ ...
├─ experiment.md
└─ config.json

ND2/search.py
├─ --checkpoint_path  ← Para cargar (opcional)
└─ --save_checkpoint  ← Para guardar (opcional)
```

Flujo de Control: 4 Casos

Caso 1: `use_checkpoint: false` (Desactivado)

```
Iter 1:
├─ Agent diseña experimento
├─ Genera datos
├─ Ejecuta: ND2 --episodes 10000
|  ├─ MCTS comienza en árbol VACÍO
|  └─ Busca 10,000 episodios
├─ NO guarda checkpoint
└─ Termina

Iter 2:
├─ Agent diseña nuevo experimento
├─ Genera datos
├─ Ejecuta: ND2 --episodes 10000
|  ├─ MCTS comienza en árbol VACÍO (nuevamente)
|  ├─ NO carga nada
|  └─ Busca 10,000 episodios
└─ Termina

Resultado: Cada iteración empieza de cero
           Pierde información anterior (comportamiento standard ND2)
```

Caso 2: `use_checkpoint: true` + `checkpoint_mode: "auto"` (Continuación automática)

```
Iter 1 (episodes=5000):
├─ Agent diseña experimento
├─ Ejecuta: ND2 --episodes 5000
|  ├─ MCTS comienza vacío
|  └─ Busca 5,000 episodios
|     Mejor: ((x*(2.01*p_curr))-p_prev1)) → R²=0.9920
├─ Guarda checkpoint automáticamente:
|  └─ iter001/mcts_checkpoint.pkl
|     (contiene: árbol, episode_count=5000, best_model, etc)
└─ Termina

Agent analiza: "R²=0.9920 es bueno pero podría mejorar con más episodios"

Iter 2 (episodes=+5000):
├─ Agent diseña experimento (mismos parámetros)
├─ Ejecuta: ND2 --episodes 10000 \
|           --checkpoint_path iter001/mcts_checkpoint.pkl
```

```
|   ├─ MCTS carga árbol anterior
|   ├─ episode_count = 5000 (donde quedó)
|   └─ Busca desde episodio 5001 hasta 10000
|      (5000 nuevos episodios refinando alrededor de lo descubierto)
|      Mejor: ((x*(2.00*p_curr))-p_prev1)) → R²=0.9999
├─ Guarda checkpoint actualizado:
|   └─ iter002/mcts_checkpoint.pkl
|      (contiene: árbol más grande, episode_count=10000, mejor_model)
└─ Termina

Resultado: Dos corridas = 10,000 episodios totales
           Sin desperdicio
           Árbol crece continuamente
           Expresión se refina progresivamente
```

Caso 3: `use_checkpoint: true` + `checkpoint_mode: "manual"` (Arranque manual)

```
Usuario define en config.json:
{
  "use_checkpoint": true,
  "checkpoint_mode": "manual",
  "initial_expression": "((x*(2.01*p_curr))-p_prev1))"
}

Iter 1:
├─ Agent diseña experimento
├─ Ejecuta: ND2 --episodes 10000 \
|           --initial_expression "((x*(2.01*p_curr))-p_prev1))"
|   ├─ MCTS comienza con expresión manual como semilla
|   ├─ Evalúa esa expresión: R²=0.992 (sin buscar, solo evaluación)
|   ├─ Agrega nodo al árbol
|   └─ Busca 10,000 episodios:
|      ├─ Episodios 1-500: Exploran alrededor de la semilla
|      ├─ Episodios 500-5000: Refinan coeficientes
|      └─ Episodios 5000-10000: Buscan alternativas
|      Mejor: ((x*(1.99*p_curr))-(1.00*p_prev1)) → R²=0.9999
├─ Guarda checkpoint:
|   └─ iter001/mcts_checkpoint.pkl
└─ Termina

Resultado: ND2 NO desperdicia episodios re-descubriendo
           Arranca desde punto conocido
           Explora eficientemente alrededor
```

Caso 4: Hybrid (Auto + Manual fallback)

```
Iter 1:
├─ checkpoint_mode: "auto"
├─ No hay checkpoint anterior
└─ ND2 busca desde cero

Iter 2:
├─ checkpoint_mode: "auto"
├─ Carga checkpoint de Iter 1 → Continúa
└─ Guarda checkpoint Iter 2

Iter 3:
├─ Usuario decide: "Cambio a manual"
├─ Modifica config.json:
│  ├─ checkpoint_mode: "manual"
│  └─ initial_expression: "((x*(1.99*p_curr))-(1.00*p_prev1))"
├─ ND2 IGNORA checkpoint de Iter 2
├─ Arranca desde expresión manual
└─ Nuevo árbol, pero con expresión "caliente"

Resultado: Flexibilidad total
           Usuario puede tomar control cuando quiera
```

Detalles de Implementación

1. Modificación `config.json`

```json
```

```json
{
  "objective": "Discover: target = (2*l_order + 1)*x*p_curr - l_order*p_prev1",

  "success_criteria": {
    "min_r2": 0.99,
    "max_complexity": 12
  },

  "constraints": {
    "max_iterations": 20,
    "max_time_per_exp": 300,
    "max_total_time": 86400
  },

  "hyperparameter_space": {
    "l_ranges": [[2,25], [8,18], [10,16], [12,14]],
    "episodes": [10000, 20000, 50000, 100000],
    "beam_sizes": [5, 10, 15, 20, 30],
    "num_samples": [500, 1000, 2000, 3000]
  },

  "checkpoint_settings": {
    "use_checkpoint": true,
    "checkpoint_mode": "auto",
    "initial_expression": null,
    "max_continuation_iterations": 3,
    "r2_threshold_for_continuation": 0.98
  },

  "agent_settings": {
    "claude_model": "claude-opus-4-20250805",
    "verbose": true
  },

  "external_paths": {
    "nd2_repo": "../ND2"
  }
}
```

Campos de control:

- `use_checkpoint: true/false` ← Activar/desactivar feature
- `checkpoint_mode: "auto" | "manual"` ← Comportamiento
- `initial_expression` ← Campo para entrada manual (null = no usar)
- `r2_threshold_for_continuation` ← Cuándo decidir continuar

2. Modificación `ND2/search/mcts.py` (30 líneas)

```python
```

```python
import pickle

class MCTS:
    def __init__(self, ...):
        # ... código existente ...
        self.episode_count = 0  # Contador de episodios

    def save_checkpoint(self, path):
        """Guardar estado MCTS completo"""
        checkpoint = {
            'tree': self.tree,
            'episode_count': self.episode_count,
            'best_model': self.best_model,
            'best_metric': self.best_metric,
            'beam_size': self.beam_size,
            'timestamp': datetime.now().isoformat()
        }
        with open(path, 'wb') as f:
            pickle.dump(checkpoint, f)
        logger.note(f"Checkpoint saved at episode {self.episode_count}")

    @classmethod
    def load_checkpoint(cls, path, rewarder, ndformer):
        """Cargar estado MCTS desde checkpoint"""
        with open(path, 'rb') as f:
            checkpoint = pickle.load(f)

        mcts = cls(rewarder=rewarder, ndformer=ndformer, ...)
        mcts.tree = checkpoint['tree']
        mcts.episode_count = checkpoint['episode_count']
        mcts.best_model = checkpoint['best_model']
        mcts.best_metric = checkpoint['best_metric']

        logger.note(f"Checkpoint loaded from episode {mcts.episode_count}")
        return mcts

    def fit(self, episode_limit=1000000, ...):
        """fit() automáticamente continúa desde episode_count anterior"""
        logger.note(f"Starting from episode {self.episode_count}, "
                    f"running until episode {episode_limit}")

        for episode in range(self.episode_count, episode_limit):
            # MCTS iteration normal
            ...
            self.episode_count += 1
```

3. Modificación `ND2/search.py` (20 líneas)

```python
parser.add_argument('--checkpoint_path', type=str, default=None,
                    help='Load MCTS checkpoint to continue search')
parser.add_argument('--save_checkpoint', type=str, default=None,
                    help='Save MCTS checkpoint after search')
parser.add_argument('--initial_expression', type=str, default=None,
                    help='Initial symbolic expression to seed MCTS')

# En main():
if args.checkpoint_path:
    logger.note(f"Loading checkpoint from {args.checkpoint_path}")
    est = MCTS.load_checkpoint(args.checkpoint_path, rewarder, ndformer)
else:
    est = MCTS(rewarder=rewarder, ndformer=ndformer, ...)

    # Seed con expresión inicial si está disponible
    if args.initial_expression:
        logger.note(f"Seeding with initial expression: {args.initial_expression}")
        from ND2.GDExpr import GDExpr
        try:
            prefix = GDExpr.str2prefix(args.initial_expression)
            reward = rewarder(prefix)
            est.best_model = prefix
            est.best_metric = reward
        except Exception as e:
            logger.warning(f"Could not load initial expression: {e}")

est.fit(...)

# Guardar checkpoint si está configurado
if args.save_checkpoint:
    est.save_checkpoint(args.save_checkpoint)
```

---

4. Modificación `autonomous_research_agent_v2.py` (50 líneas)

```python
```

```python
class ResearchAgentV2:

    def _decide_checkpoint_strategy(self, iteration):
        """Decide si usar checkpoint y cómo"""

        checkpoint_config = self.config.get("checkpoint_settings", {})
        use_checkpoint = checkpoint_config.get("use_checkpoint", False)
        checkpoint_mode = checkpoint_config.get("checkpoint_mode", "auto")
        initial_expr = checkpoint_config.get("initial_expression")

        if not use_checkpoint:
            # Feature desactivado
            return None, None

        if checkpoint_mode == "manual" and initial_expr:
            # Modo manual: usar expresión definida
            return "manual", initial_expr

        if checkpoint_mode == "auto" and iteration > 1:
            # Modo automático: cargar checkpoint anterior
            prev_checkpoint = self.results_dir / f"iter{iteration-1:03d}" / "mcts_ch
            if prev_checkpoint.exists():
                return "auto", str(prev_checkpoint)

        # Por defecto: sin checkpoint
        return None, None

    def run_autonomous_loop(self, max_iterations=20):

        for iteration in range(1, max_iterations + 1):

            # Determinar estrategia checkpoint
            checkpoint_mode, checkpoint_path = self._decide_checkpoint_strategy(iter

            # ... design experiment ...

            # Construir comando ND2 con checkpoint si aplica
            cmd = [
                "python", f"{self.nd2_path}/search.py",
                "--name", exp_name,
                "--data", data_path,
                "--vars", "l_order", "x", "p_prev1", "p_prev2", "p_curr",
                "--target_var", "target",
                "--episodes", str(hyperparams['episodes']),
                "--beam_size", str(hyperparams['beam_size']),
            ]
```

```
        if checkpoint_mode == "auto" and checkpoint_path:
            self._log(f"[Iter {iteration}] Continuing from checkpoint")
            cmd.extend(["--checkpoint_path", checkpoint_path])

        elif checkpoint_mode == "manual" and checkpoint_path:
            self._log(f"[Iter {iteration}] Seeding with manual expression")
            cmd.extend(["--initial_expression", checkpoint_path])

        else:
            self._log(f"[Iter {iteration}] Starting fresh (no checkpoint)")

        # Guardar checkpoint después si está habilitado
        checkpoint_config = self.config.get("checkpoint_settings", {})
        if checkpoint_config.get("use_checkpoint"):
            iter_checkpoint = f"{self.results_dir}/iter{iteration:03d}/mcts_chec
            cmd.extend(["--save_checkpoint", iter_checkpoint])

        # Ejecutar
        result = self.executor.run(exp_name, data_path, hyperparams, str(self.re

        # ... análisis, guardar resultados ...

        # Decidir si continuar (lógica del agente)
        r2 = result.get('best_r2', 0)
        r2_threshold = checkpoint_config.get("r2_threshold_for_continuation", 0.

        if r2 > 0.99:
            self._log("✓ Success criteria met, stopping")
            break

        elif r2 > r2_threshold and checkpoint_config.get("use_checkpoint"):
            self._log(f"⚠ R²={r2:.4f} is good but could improve, will continue i

        else:
            self._log(f"✗ R²={r2:.4f} is low")
```

Flujo de Usuario: Casos de Uso

Uso Caso 1: Agent decide automáticamente (despreocupado)

```
config.json:
{
  "checkpoint_settings": {
```

```
    "use_checkpoint": true,
    "checkpoint_mode": "auto",
    "r2_threshold_for_continuation": 0.98
  }
}

Entonces:
- Iter 1: Busca 10000 episodios → R²=0.98 → Guarda checkpoint
- Iter 2: Carga checkpoint → Busca +10000 episodios → R²=0.995 → Guarda checkpoint
- Iter 3: Carga checkpoint → Busca +10000 episodios → R²=0.9999 → Guarda
checkpoint
- Iter 4: Agent decide parar

Usuario: Durmiendo 😴
```

Uso Caso 2: Usuario interviene manualmente

```
Iter 1: Descubre ecuación X (buena pero no perfecta)
Usuario: "Quiero partir de esta expresión específica y refinarla"

Modifica config.json:
{
  "checkpoint_settings": {
    "use_checkpoint": true,
    "checkpoint_mode": "manual",
    "initial_expression": "((x*(2.01*p_curr))-p_prev1))"
  }
}

Iter 2: ND2 arranca ahí y refina
```

Uso Caso 3: Desactivar checkpoint (modo tradicional)

```
config.json:
{
  "checkpoint_settings": {
    "use_checkpoint": false
  }
}

Entonces:
- Cada iteración comienza de cero
- Comportamiento estándar ND2
- Pierde información anterior
- Pero funciona perfectamente
```

Visualización en `experiment.md`

```markdown
## DISCOVERIES

### Iteration 1
**Status**: SUCCESS
**R²**: 0.9894
**Complexity**: 7
**Equation**: `((x*(2.0134*p_curr))-p_prev1)`
**Checkpoint saved**: YES → iter001/mcts_checkpoint.pkl
**Episodes used**: 5000 (total: 5000)

---

### Iteration 2
**Status**: SUCCESS
**R²**: 0.9920
**Complexity**: 7
**Equation**: `((x*(2.0078*p_curr))-(0.9998*p_prev1))`
**Checkpoint source**: Loaded from iter001
**Checkpoint saved**: YES → iter002/mcts_checkpoint.pkl
**Episodes used**: +5000 (total: 10000)
**Analysis**: Continuing from previous checkpoint. Refined coefficients.

---

### Iteration 3
**Status**: SUCCESS
**R²**: 0.9999
**Complexity**: 7
**Equation**: `((x*(2.00*p_curr))-(1.00*p_prev1))`
**Checkpoint source**: Loaded from iter002
**Checkpoint saved**: YES → iter003/mcts_checkpoint.pkl
**Episodes used**: +10000 (total: 20000)
**Analysis**: ✓ SUCCESS CRITERIA MET
```

Tabla de Control

Parámetro	Valor	Comportamiento
`use_checkpoint`	`false`	Cada iter: CERO. Pierde info. (Standard ND2)
`use_checkpoint`	`true` + `mode: auto`	Continúa de anterior si existe. Si no, empieza cero
`use_checkpoint`	`true` + `mode: manual`	Ignora anterior. Arranca de `initial_expression`
`initial_expression`	`null`	Sin expresión manual
`initial_expression`	`"((x*2)-y)"`	ND2 arranca con esa semilla

Resumen: Qué se logra

Feature	Activado	Desactivado
Checkpoint	Guarda estado MCTS después de cada run	No guarda nada
Continuación automática	Si R^2 es "regular", continúa en Iter N+1	Cada Iter es independiente
Expresión manual	Usuario puede ingresar expresión en config	Config ignorada
Flexibilidad	Usuario puede cambiar config entre iteraciones	Una config fija
Eficiencia	Reutiliza búsqueda anterior	Desperdicia esfuerzo

Checklist de Implementación

- [ ] **Modificar `config.json` template** - Agregar campos checkpoint_settings
- [ ] **Modificar `ND2/search/mcts.py`** - save_checkpoint() y load_checkpoint()
- [ ] **Modificar `ND2/search.py`** - Argumentos CLI y lógica de carga/seed
- [ ] **Modificar `autonomous_research_agent_v2.py`** - Decidir cuándo usar checkpoint
- [ ] **Actualizar `experiment.md`** - Reportar si checkpoint fue usado
- [ ] **Documentar en README** - Explicar cómo usar feature
- [ ] **Probar casos:** 1) disabled, 2) auto, 3) manual
- [ ] **Validar compatibilidad** - Sin checkpoint debe funcionar como antes

Tiempo de Implementación

Componente	Tiempo	Dificultad
Modificar `config.json`	10 min	Trivial
Modificar MCTS (save/load)	30 min	Fácil
Modificar search.py (args + logic)	20 min	Fácil
Modificar agent	30 min	Medio
Testing	30 min	Medio
Total	**2 horas**	-

¿Te parece este plan? ¿Modifico algo antes de empezar la implementación?